



LinkedIn Clone: LinkedIn-like платформа

Frontend-частина дипломного проекту професійного нетворкінгу

- Формат звіту: кожен слайд пояснює причину вибору рішення та результат
- Проект: сайт для професійної взаємодії, пошуку роботи та комунікації
- Frontend розробники: Ямчук Тимур, Андрій Ротарь
- Стек frontend: React 19, Vite 6, React Router, Context API
- Демо-режим: social demo та mock-дані для стабільної демонстрації на захисті

1. Звіт: Вступ: чому саме LinkedIn

Обґрунтування рішення: LinkedIn обрано як найсильнішу модель, бо він поєднує професійний профіль, мережу контактів, контент, вакансії та комунікацію.

6

основних розділів об'єднані в одному сайті

- Причина вибору: Це не просто сайт з вакансіями, а повна професійна екосистема для кандидата, роботодавця і спільнот.
- Обране рішення: Такий формат дозволяє показати більше дипломної роботи: UI, маршрути, клієнтські стани та користувацькі сценарії.
- Реалізація: Клон LinkedIn добре демонструється на захисті через demo-flow: вхід → профіль → стрічка → вакансії → чат.

2. Звіт: Файлова структура проєкту

Звітний огляд: Фокус доповіді — frontend SPA; backend показано лише для розуміння масштабу системи

frontend/ — наш фокус

```
frontend/  
├── src/  
│   ├── app/ router, providers, layout  
│   ├── pages/ auth, home, network, jobs, chat, admin  
│   ├── features/ auth, profile, jobs, chat, network  
│   └── shared/  
│       ├── api/ HTTP-клієнт, paths  
│       ├── ui/ спільні компоненти  
│       └── lib/ storage, session  
├── vite.config.js  
└── package.json
```

backend/ — контекст

```
backend/ (контекст масштабу)  
├── Identity · Profile · Professional  
├── Network · Content · Messaging  
├── Jobs · Events · Notifications  
└── Facade.API
```

Структура з реального репозиторію: React SPA розбито на pages, features, app та shared.

3. Звіт: Порівняння з конкретними конкурентами

Стан у конкурентів: Що є у Work.ua, LinkedIn, Djinni, Robota.ua

- Аналіз: Work.ua: сильний пошук вакансій, фільтри, резюме та відгуки, але фокус переважно на job-board сценарії.
- Аналіз: LinkedIn: сильна глобальна мережа, профіль, контент і вакансії, але продукт перевантажений для локального навчального MVP.
- Аналіз: Djinni: зручний IT-рекрутинг і анонімний профіль, але вузький фокус на наймі без широких спільнот і стрічки.
- Аналіз: Robota.ua: багато вакансій, резюме і роботодавців, але менше соціальної взаємодії, чатів і професійного контенту.

Наше рішення: Чого бракує там, але є у LinkedIn Clone

- Перевага: У LinkedIn Clone вакансії не ізольовані: вони пов'язані з профілем, чатами, контактами і загальним шляхом кандидата.
- Перевага: LinkedIn Clone бере логіку LinkedIn, але показує її у компактному MVP, де кожен модуль зрозумілий на захисті.
- Перевага: У LinkedIn Clone є не тільки найм, а й контент, groups/pages/events, глобальний пошук і особистий кабінет.
- Перевага: Наш сайт об'єднує job-board, професійну соцмережу і внутрішню комунікацію в одному frontend-сценарії.

4. Звіт: Переваги UX нашого сайту

Звітний висновок: Акцент на зручності інтерфейсу для користувача, а не на backend-інтеграції



Єдиний UX

Рішення: одна навігація для всіх сценаріїв



Dark/Light

Рішення: перемикання теми через CSS variables



i18n

Рішення: 4 мови інтерфейсу в runtime



Картки

Рішення: зрозуміла структура контенту



Анімації

Рішення: плавні переходи між сторінками



Пошук

Рішення: глобальний пошук по сутностях



Демо-режим

Рішення: social demo без реєстрації по пошті



Fallback

Рішення: стабільні mock-дані для захисту



Для кандидата: одна навігація, теми, мови та social demo — все в одному UX.

5. Звіт: Що є у конкурентів і що додано у нас

Стан у конкурентів: У конкурентів

- Аналіз: Вакансії існують окремо від соціальної активності кандидата.
- Аналіз: Пошук роботи часто не пов'язаний з чатами та мережею контактів.
- Аналіз: Демонстраційні проєкти нерідко залишаються статичними макетами.
- Аналіз: Інтерфейс не завжди показує повний шлях користувача від входу до дії.

Наше рішення: У нашому LinkedIn Clone

- Перевага: Користувач шукає роботу, веде профіль, читає контент і спілкується в одному місці.
- Перевага: Вакансії, контакти та повідомлення об'єднані в логічний кандидатський сценарій.
- Перевага: Frontend має demo-режим і fallback-дані для гарантованої демонстрації на захисті.
- Перевага: На захисті показуємо послідовність через social demo: auth → profile → feed → jobs → chat.

6. Звіт: Компонентна структура frontend

Звітний огляд: Кожен блок — ізольований React-модуль з власною логікою

Дерево папок frontend

```
frontend/  
├── src/  
│   ├── app/ router, providers, layout  
│   ├── pages/ auth, home, network, jobs, chat, admin  
│   ├── features/ auth, profile, jobs, chat, network  
│   ├── shared/  
│   │   ├── api/ HTTP-клієнт, paths  
│   │   ├── ui/ спільні компоненти  
│   │   └── lib/ storage, session  
├── vite.config.js  
└── package.json
```

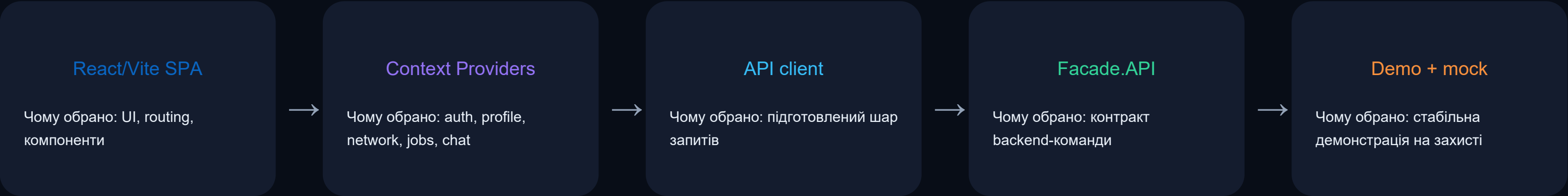
Ізольовані React-модулі

- ✓ Модуль: pages/ — екрани за сценаріями: auth, home, network, jobs, chat, admin
- ✓ Модуль: features/ — бізнес-логіка модулів, API-виклики, mappers
- ✓ Модуль: app/ — router, providers, layout, ініціалізація UI
- ✓ Модуль: shared/ — api, ui, lib: спільний код без прив'язки до одного модуля
- ✓ Модуль: contexts — Auth, Profile, Network, Vacancies, Chat (глобальний стан)

Архітектура: Код розбито за папками так, щоб модулі не залежали один від одного напряму.

7. Звіт: Архітектура frontend

Frontend: Context API, API-клієнт, демо-режим



Результат рішення: Frontend керує станом через Context API та API-клієнт; backend і БД — зона backend-команди, ми показуємо готовий клієнтський шар.

8. Звіт: Маршрутизація frontend

SPA з guarded routes, state providers, темою та i18n

/auth

Роль у рішенні: публічний маршрут, social demo

RequireAuth

Роль у рішенні: захист приватних сторінок

AuthBootstrapGate

Роль у рішенні: ініціалізація сесії при завантаженні

/home ... /profile

Роль у рішенні: приватні маршрути основних модулів

PageTransitionOutlet

Роль у рішенні: анімації переходів між сторінками

redirect / → /home

Роль у рішенні: стартовий сценарій після входу

9. Звіт: Demo-flow для захисту

Демонстраційний шлях користувача на захисті



Social demo

Профіль

Стрічка

Спільноти

Вакансії

Чати

Звітний результат: На захисті використовуємо social demo і mock-дані — без live-реєстрації та без залежності від нестабільного API.

10. Звіт: Адмін-панель платформи

Звітний огляд: Окремий UI-контур /admin для модерації та керування LinkedIn Clone

Доступ і архітектура

- ✓ /admin/* → RequireAuth + RequireAdmin
- ✓ JWT role Admin (user.isAdmin)
- ✓ Без ролі — AdminForbiddenPage (403)
- ✓ AdminLayout: sidebar з 7 розділами
- ✓ adminApi.js → /api/admin/*

Розділи адмін-панелі

DB	Dashboard	stats: users, posts, vacancies, events	✓
US	Users	list, filters, lock/unlock, roles, create user	✓
CT	Content	moderation постів: search, delete/restore	✓
CM	Comments	moderation коментарів до постів	✓
JB	Jobs	moderation вакансій + recommended queries	✓
EV	Events	moderation подій: delete/restore	✓
RL	Roles	ролі User/Admin у drawer користувача	✓

Результат: UI адмінки на frontend; demo/mock — порожні дані; повний сценарій — Admin JWT + backend.

11. Звіт: Розподіл frontend-роботи

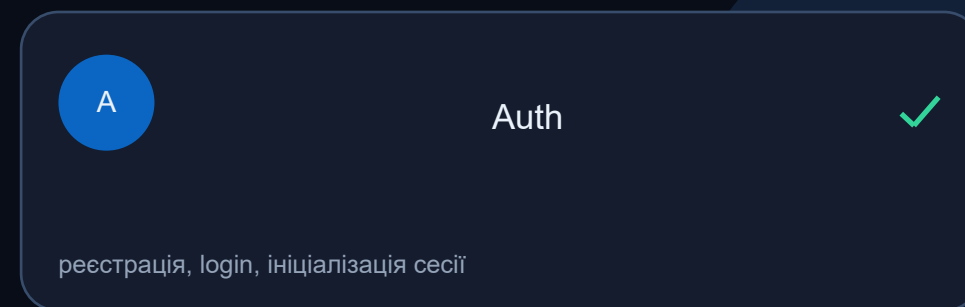
Ямчук Тимур

- Зона відповідальності: Вікно реєстрації та auth UX.
- Зона відповідальності: Сторінка вакансій і пошук роботи.
- Зона відповідальності: Чати, дизайн і особистий кабінет.

Андрій Ротарь

- Зона відповідальності: Головний екран і динамічна контентна стрічка (Feed).
- Зона відповідальності: Архітектура Network: контакти, groups, pages, events.
- Зона відповідальності: API-клієнт, демо-режим і підготовка до backend-інтеграції.

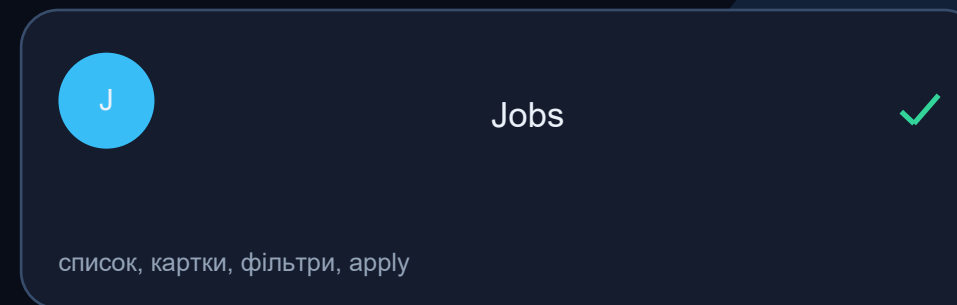
12. Звіт: Ямчук Тимур: вікно реєстрації



Обґрунтування рішення: Реєстрація є першою точкою входу користувача в платформу.

- Причина вибору: Побудовано зрозумілий auth UX: користувач бачить форму, стани введення та результат дії.
- Обране рішення: Опрацьовано валідацію полів і повідомлення про помилки, щоб сценарій не виглядав як статичний макет.
- Реалізація: Після успішної реєстрації користувач переходить до подальшого користування сайтом.
- Результат: Auth-екран стилістично узгоджено з іншими частинами платформи.

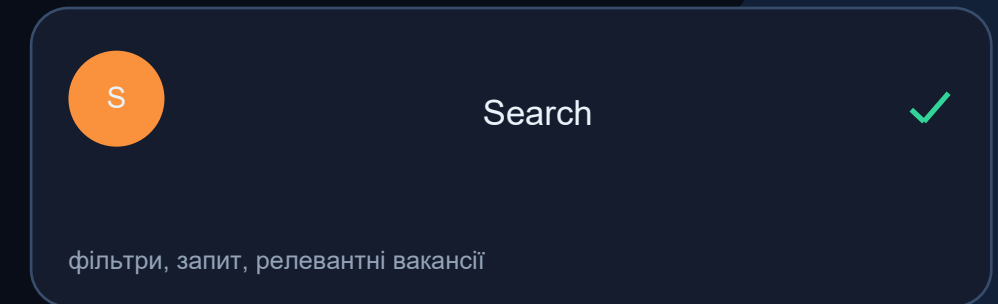
13. Звіт: Ямчук Тимур: сторінка вакансій



Обґрунтування рішення: Сторінка вакансій реалізує один з головних сценаріїв LinkedIn-like платформи.

- Причина вибору: Створено список вакансій з картками, основними параметрами і зрозумілою структурою перегляду.
- Обране рішення: Додано відображення формату роботи, рівня досвіду, локації та інших характеристик вакансії.
- Реалізація: Підготовлено сценарії збереження вакансії та відгуку користувача.
- Результат: Сторінка вписана в загальну навігацію і не виглядає окремим модулем.

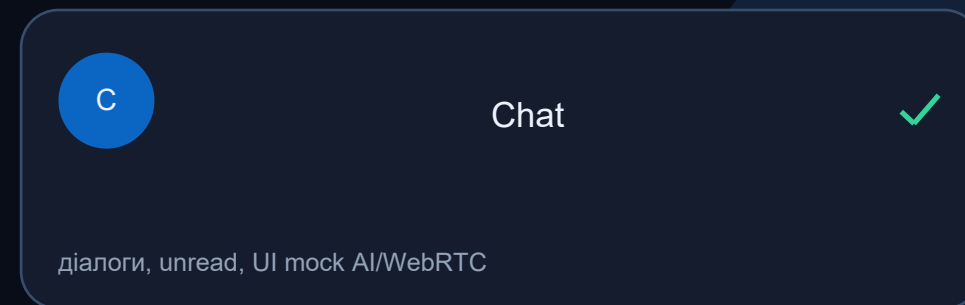
14. Звіт: Ямчук Тимур: пошук роботи



Обґрунтування рішення: Пошук роботи робить модуль вакансій корисним для щоденного сценарію кандидата.

- Причина вибору: Реалізовано фільтрацію вакансій за ключовими параметрами.
- Обране рішення: Опрацьовано сценарій, коли результатів немає або користувач змінює запит.
- Реалізація: Пошук допомагає швидко перейти від загального списку до релевантних вакансій.
- Результат: Цей блок підсилює головну перевагу сайту: вакансії пов'язані з профілем і комунікацією.

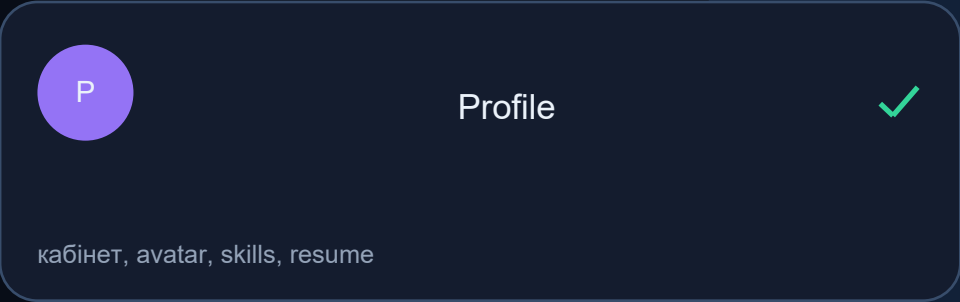
15. Звіт: Ямчук Тимур: чати



Обґрунтування рішення: Чати додають платформі внутрішню комунікацію без переходу в сторонні месенджери.

- Причина вибору: Створено інтерфейс діалогів, повідомлень і станів unread.
- Обране рішення: Додано сценарії архівації, mute, видалення повідомлень і share posts.
- Реалізація: UI-шар для AI-підказок і майбутнього WebRTC — це mock-моделі, не production-функції.
- Результат: Комунікація підтримує ідею LinkedIn-like продукту: робота, контакти і повідомлення в одному місці.

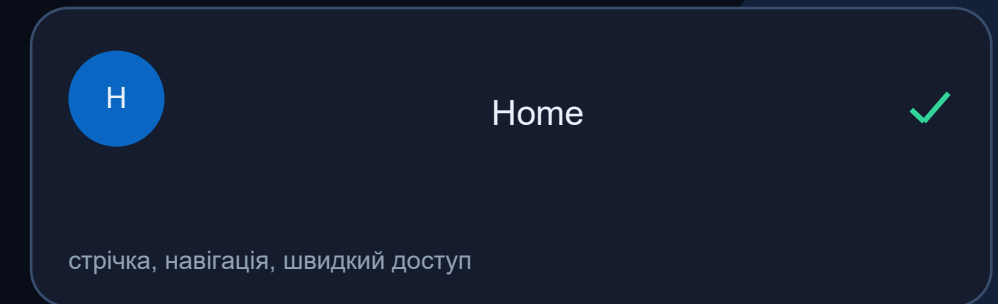
16. Звіт: Ямчук Тимур: дизайн і особистий кабінет



Обґрунтування рішення: Особистий кабінет і дизайн формують завершене враження від frontend-частини.

- Причина вибору: Профіль користувача містить особисті та професійні дані, avatar, skills і resume.
- Обране рішення: Візуальний стиль узгоджено між auth, vacancies, chat і profile.
- Реалізація: Кабінет працює як центр професійної ідентичності користувача.
- Результат: Дизайн побудований на картках, темній/світлій темі, акцентах і зрозумілій навігації.

17. Звіт: Андрій Ротарь: головний екран



Обґрунтування рішення: Головний екран задає перше враження і веде користувача до основних розділів платформи.

- Причина вибору: Створено структуру home-сторінки з фокусом на професійний контент.
- Обране рішення: Додано швидкий доступ до мережі, вакансій, повідомлень і профілю.
- Реалізація: Головний екран працює як стартова точка після авторизації.
- Результат: Навігація допомагає показати сайт як цілісну платформу.

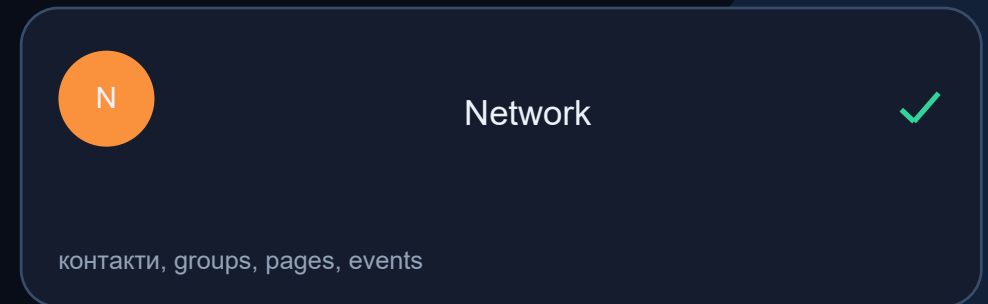
18. Звіт: Андрій Ротарь: контентна стрічка (Feed)



Обґрунтування рішення: Feed — найскладніша frontend-частина: динамічні пости, реакції та оновлення стану.

- Причина вибору: Реалізовано подачу постів у форматі професійної стрічки з composer і sidebar inbox.
- Обране рішення: Додано обробку реакцій, коментарів і оновлення UI після дій користувача.
- Реалізація: Стрічка синхронізується з providers і mock/API-даними без перезавантаження сторінки.
- Результат: Це технічно складніший модуль, ніж статичні форми auth або profile.

19. Звіт: Андрій Ротарь: архітектура Network



Обґрунтування рішення: Network поєднує контакти, підписки, groups, pages і events в одній вкладці.

- Причина вибору: Побудовано структуру вкладки network з окремими підрозділами та навігацією.
- Обране рішення: Реалізовано зв'язки між контактами, профілями та соціальними сутностями.
- Реалізація: Groups/pages/events підсилюють відмінність LinkedIn Clone від звичайного job-board.
- Результат: Модуль показує архітектурну складність frontend, а не лише візуальний дизайн.

20. Звіт: Андрій Ротарь: API-клієнт і демо-режим

A

Арі-Клієнт

✓

арі-клієнт і демо-режим

Обґрунтування рішення: На захисті — демо-режим з fallback; API-клієнт готовий до підключення backend.

- Причина вибору: Social demo і fallback-дані гарантують стабільну демонстрацію без залежності від сервера.
- Обране рішення: Створено структуру API-клієнта та проху для майбутньої backend-інтеграції.
- Реалізація: Механізм оновлення сесії є в коді, але live refresh на захисті не показуємо.
- Результат: Backend-баги — зона backend-команди; frontend-шар готовий до підключення.

21. Звіт: Висновок по frontend-роботі

Причина / виконання: Що реалізовано

- Звіт: Завершений frontend MVP: UI, routing, providers, demo-flow і компонентна архітектура.
- Звіт: Два зони відповідальності: кандидатський UX (Тимур) і контент/мережа/API-клієнт (Андрій).
- Звіт: Демо-режим забезпечує стабільну презентацію без ризику live API-помилки.

Результат / висновок: Практичний результат

- Висновок: Це не макет, а архітектурно завершений frontend-продукт з реальними сценаріями.
- Висновок: Проект показує перевагу LinkedIn-like підходу над простими job-board.
- Висновок: Frontend готовий до подальшого підключення backend після стабілізації API.



22. Звіт: Підсумок

Frontend MVP з demo-flow — архітектурно завершений продукт для захисту

Підсумковий висновок: LinkedIn Clone — це frontend MVP професійної екосистеми, а не окремий job-board.

Підсумковий висновок: На захисті демонструємо demo-flow через social demo та mock-дані.

Підсумковий висновок: Головний результат — архітектурно завершений клієнтський шар і зрозумілий UX.

